

```
rev('abcdef')
  ↓
return rev('bcdef', 'a' + '')
  ↓
return rev('cdef', 'b' + 'a')
  ↓
return rev('def', 'c' + 'ba')
  ↓
return rev('ef', 'd' + 'cba')
  ↓
return rev('f', 'e' + 'dcba')
  ↓
return rev('', 'f' + 'edcba')
  ↓
return 'fedcba'
```

Figure 8-2: The process of transformations that `rev('abcdef')` makes to the string `fedcba`

Tail Recursive Find Substring

Some recursive functions naturally end up using the tail recursion pattern. If you look at the `findSubstringRecursive()` function in the *findSubstring.py* and *findSubstring.html* programs in Chapter 2, you'll notice that the last action for the recursive case is to return the value of the recursive function call. No adjustments are needed to make this function tail recursive.

Tail Recursive Exponents

The *exponentByRecursion.py* and *exponentByRecursion.html* programs, also from Chapter 2, are not good candidates for tail recursion. These programs have two recursive cases for when the `n` parameter is even or odd. This is fine: as long as all the recursive cases return the return value of the recursive function call as their last action, the function can use tail call optimization.